Conceptual Model, Version 2.0

Ontology-Guided Agentic Capabilities Orchestration (AC-001)

Team AgenticCapabilities · Ontology Engineering 2026, Assignment 6 · Version 2.0, October 15, 2026
Gia Oriana Santos, Barbara Kotlan, Jiaying (Michael) Wang, Jialin (Gary) Fang, Elioenai Lincon Guedes Alves

This model covers the orchestration layer of use case AC-001: what a user request asks for, which capabilities, skills and tools can meet it, how candidate orchestration plans are built and compared, and what the knowledge graph (KG) records when a plan runs. Version 2.0 is aligned with the team ontology OE_6_TeamAgenticCapabilities.rdf: every class, is-a line, labeled arrow and attribute in the 13 diagrams is a class, subclass axiom, object property or datatype property of the ontology, with the same name, and a script checks all 273 of them against the ontology file. The ontology has 65 classes, 33 object properties, 9 datatype properties and 11 individuals of its own. Its deepest chains have four levels below a reused top class, for example ConflictingEvidenceAssessment, EvidenceAssessment, Assessment, InformationArtifact under prov:Entity. The three competency questions the ontology is scoped to, CQ 1, CQ 3 and CQ 5, are marked in the table at the end.

Notation. UML 2.5.1 class diagrams drawn with PlantUML. A box is a class; attributes inside a box are datatype properties. A line with a hollow triangle points from a subclass to its superclass (is a). A labeled arrow is an object property drawn from its domain to its range; a dashed arrow whose name starts with "/" is derived, meaning the reasoner infers it. «AgentO», «P-Plan», «PROV-O», «OWL-S», «Commons» and «BEAM» mark classes reused from those vocabularies; «defined» marks classes whose members the reasoner infers from a definition, and an «enumeration» box lists the individuals that are the members of a class. A gray box is a class detailed in another figure. Property prefixes name reused vocabularies: agento: AgentO [3], prov: PROV-O [9], process: OWL-S [8], pplan: P-Plan [12]; properties without a prefix belong to the team ontology. In an example, a box written name : Class is an individual and an orange dashed arrow is a relation the reasoner infers. Notes give axioms in Protégé's Manchester syntax.

Changes in Version 2.0. Everything new or changed since Version 1.1 is drawn in blue, and each figure ends with a blue "Changes in 2.0" paragraph. The main changes:

- Names now match the ontology. Version 1.1's satisfies is split into satisfiesRequirement and satisfiesConstraint; violates becomes violatesConstraint; a plan's steps use hasStep and OrchestrationWorkflowStep; Resource, Instructions, MCPTool and MCPServer become SkillResource, Instruction, ModelContextProtocolTool and ModelContextProtocolServer (class names are singular and have no abbreviations); the datatype properties priority, score, rank, direction, weight, value and unit become hasPriority, hasScore, hasRank, hasDirection, hasWeight, hasValue and hasUnit (property names are verbs).
- Hard and soft constraints are separated in the model, as the A5 feedback asked: violatesConstraint links a candidate plan only to a HardConstraint, a candidate that violates one is classified as rejected, and a soft constraint only lowers a plan's score and rank (Figure 6).
- Crowded figures are split, as the A5 feedback asked: Figure 3 into 3a and 3b, Figure 5 into 5a and 5b, Figure 7 into 7a and 7b, and Figure 8 into 8a, 8b and 8c. Figure 8c is new and shows how evidence is summarized and weighed.
- Superclasses that the ontology reuses are drawn: pplan:Plan, pplan:Step, pplan:Activity, prov:Entity, agento:Tool, process:Process, Commons Measure and BEAM Resource.
- All diagrams use one style and are exported at 300 dpi and as SVG.

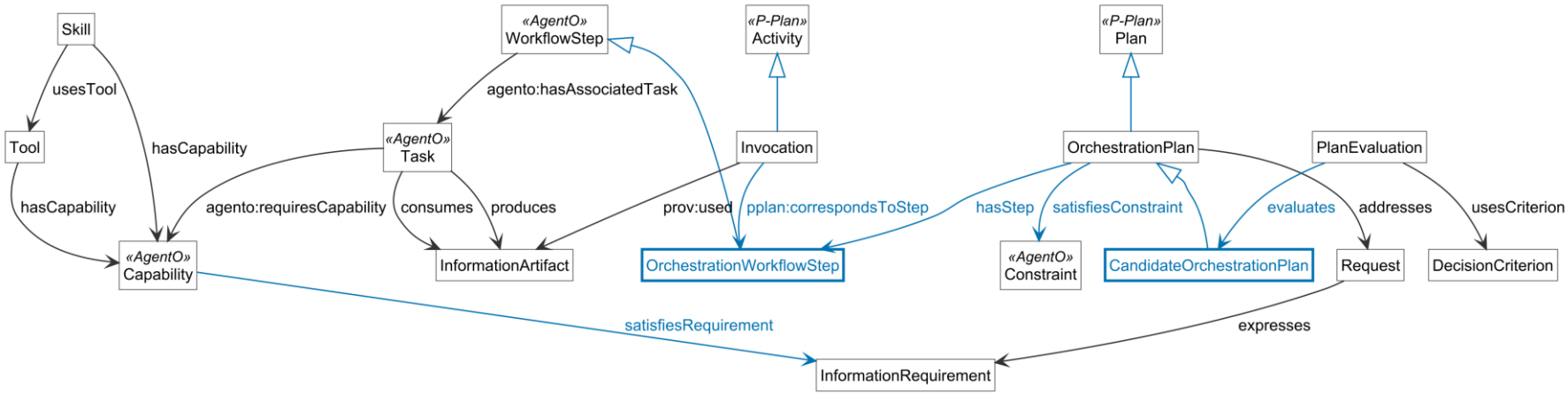


Figure 1. Overview of the core classes

Shows. The core class of each area and the relationships that tie the areas together. A Request expresses InformationRequirements, which Capabilities satisfy; Skills and Tools have Capabilities. An OrchestrationPlan addresses the Request and satisfies Constraints; its OrchestrationWorkflowSteps, a kind of AgentO WorkflowStep, are associated with Tasks, and each Task requires a Capability and consumes and produces InformationArtifacts. PlanEvaluations evaluate CandidateOrchestrationPlans on DecisionCriteria. When the plan runs, each Invocation corresponds to one OrchestrationWorkflowStep and uses InformationArtifacts.

Competency questions. The three focus questions run through this figure. Request → InformationRequirement ← Capability ← Skill answers CQ 3; OrchestrationPlan → OrchestrationWorkflowStep → Task → Capability → InformationRequirement answers CQ 5; PlanEvaluation → CandidateOrchestrationPlan answers CQ 1, with the details in Figures 5a and 6.

Changes in 2.0. satisfies is split into satisfiesRequirement (capability to requirement) and satisfiesConstraint (plan or capability to constraint), the two properties of the ontology. A plan reaches its steps through hasStep and OrchestrationWorkflowStep, a subclass of agento:WorkflowStep, which the ontology uses instead of agento:hasWorkflowStep. WorkflowStep stays in the overview: agento:hasAssociatedTask is AgentO's link from a WorkflowStep to its Task, and AgentO's StartStep and EndStep are WorkflowSteps too (Figure 5a). evaluates now ends at CandidateOrchestrationPlan, its range in the ontology, so the overview shows that class. OrchestrationPlan and Invocation show their P-Plan superclasses, and Capability is marked as AgentO's class, which the team ontology declares equivalent.

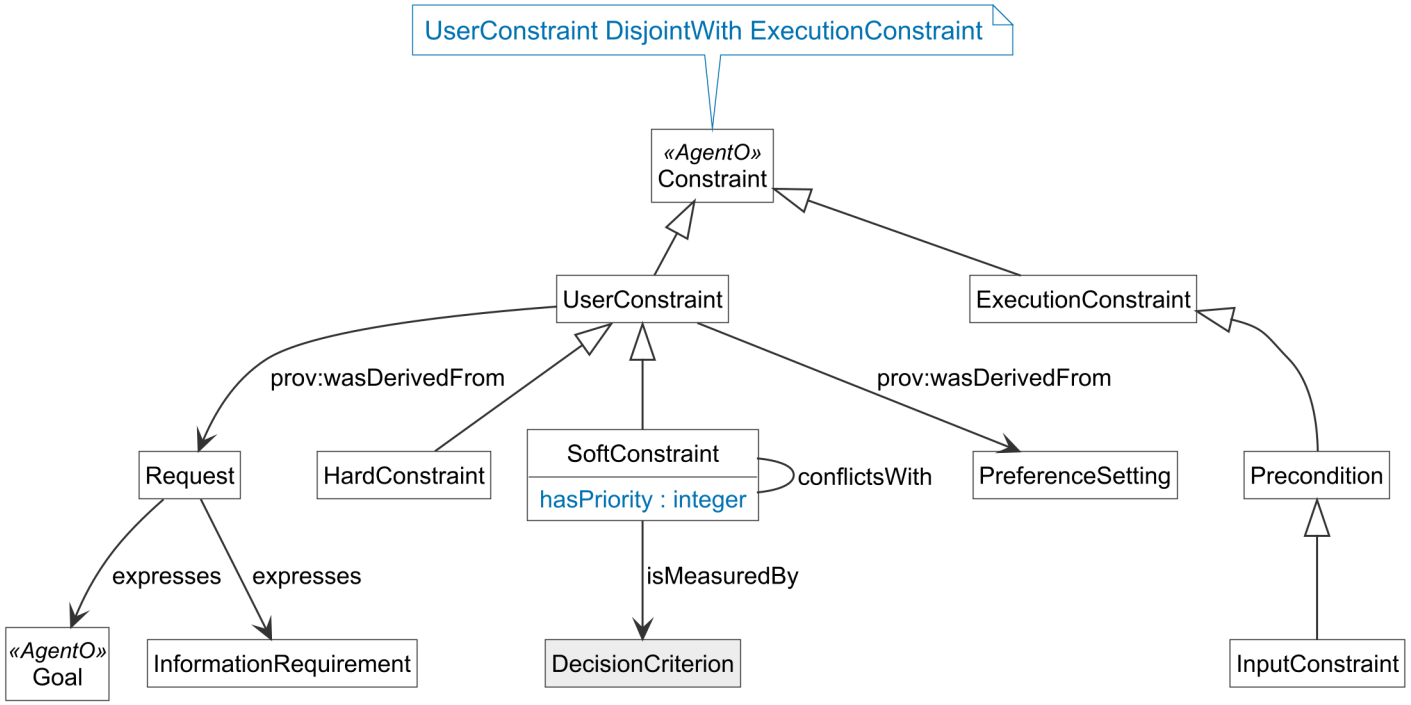


Figure 2. Requests, requirements and constraints

Shows. What the NLP tool and harness extract in basic flow steps 2–3: the Goals and InformationRequirements a Request expresses, and the UserConstraints that steer plan selection. prov:wasDerivedFrom records whether a constraint came from the Request or from the user's PreferenceSettings. PreferenceSettings are what a user saves before making requests; no step consumes or produces them, so they are not information artifacts and stay outside the artifact hierarchy of Figure 8a.

Modeling decisions. User constraints are split by how they are used. A HardConstraint must hold, for example Jessica's safety screening, which her PreferenceSettings turn on for her prescription medication; a plan that violates it is rejected. A SoftConstraint is something to optimize, for example Ken's "minimize external tool calls". Each SoftConstraint has a user-given priority (hasPriority) and is measured by a DecisionCriterion (Figure 6). conflictsWith links two soft constraints that push the same criterion in opposite directions, such as maximizing and minimizing tool use (alternate flow 3); the agent then asks the user to revise them. Constraints that only trade off, like Olivia's strong evidence and few tool calls, are not conflicts: their priorities weight them. ExecutionConstraints come from planning: a Precondition must hold before a tool or skill runs, and an InputConstraint is a precondition on one input value. A constraint comes either from the user or from planning, so UserConstraint and ExecutionConstraint are disjoint.

Competency questions. CQs 1, 3, 4, 6 and 7 start by retrieving these constraints; alternate flow 3 uses conflictsWith. In CQ 3 Jessica's safety-screening HardConstraint is derived from her PreferenceSetting.

Changes in 2.0. priority is renamed hasPriority. The disjointness of UserConstraint and ExecutionConstraint, stated in the ontology, is drawn. Constraint is marked as AgentO's class, which the ontology reuses.

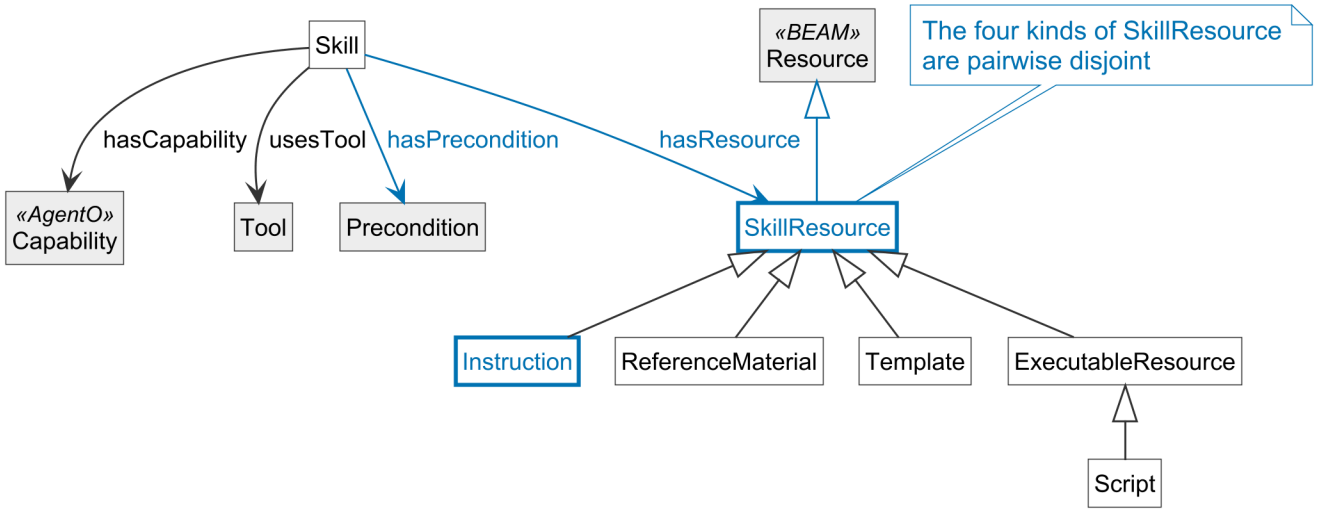


Figure 3a. Skills and skill resources

Shows. A Skill has Capabilities (Figure 3b), bundles SkillResources, uses Tools and has Preconditions. The resources are Instructions, ReferenceMaterial, Templates and ExecutableResources such as Scripts, as in the skill packages of SkillsBench [5].

Modeling decisions. SkillResource is the team's class for the parts of a skill package. It sits under BEAM's Resource, the class AgentO uses for what tasks require and produce [3], and its four kinds are disjoint. A skill's precondition uses the team's hasPrecondition: OWL-S's process:hasPrecondition has the domain process:Process and would make every skill a process. Tools keep process:hasPrecondition (Figure 4).

Competency questions. CQ 9 checks a skill's preconditions before it runs; CQ 3 and CQ 5 start from the skills' capabilities in Figure 3b.

Changes in 2.0. Figure 3 is split into 3a and 3b; the A5 feedback found crowded inheritance links in the capabilities figure. Resource is renamed SkillResource and Instructions is renamed Instruction, as in the ontology. process:hasPrecondition is replaced by the team's hasPrecondition. Skill no longer consumes and produces artifacts in this figure: in the ontology consumes and produces belong to Task, and a skill's inputs and outputs are read through the tasks it performs (performedBySkill, Figure 5a).

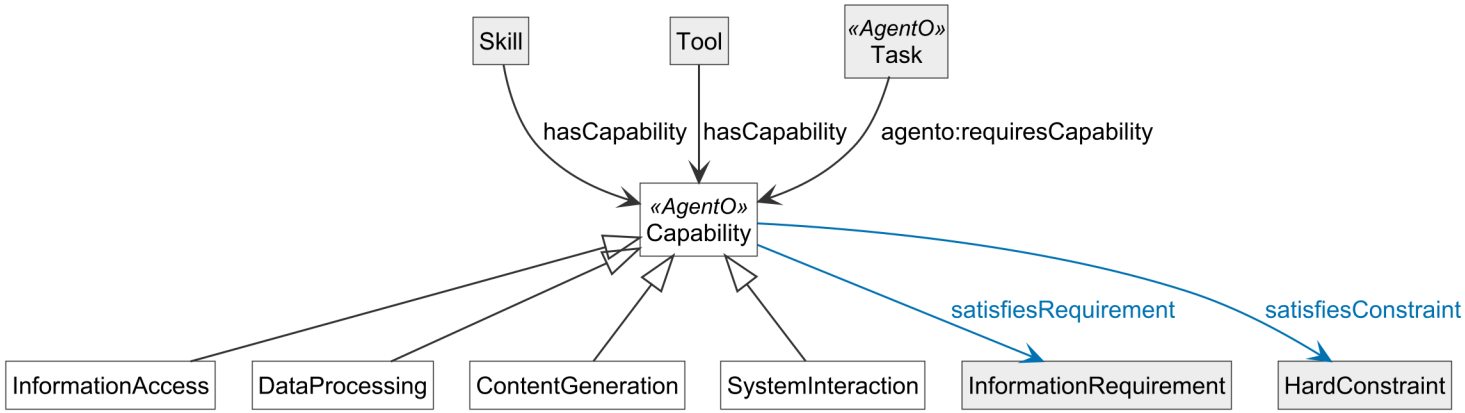


Figure 3b. Capabilities

Shows. Capabilities come in four kinds, following Mulang et al. [1]: InformationAccess, DataProcessing, ContentGeneration and SystemInteraction. Skills and Tools have Capabilities and Tasks require them. A Capability satisfies InformationRequirements and HardConstraints.

Modeling decisions. Skill is not a subclass of Capability: a skill is a packaged specification that has capabilities, and the same capability, such as screening for interactions, can come from a skill or directly from a tool. Keeping Capability separate lets the reasoner compare the two. hasCapability widens AgentO's property of the same name, whose domain is only Tool. The team's Capability is the same class as agento:Capability (equivalent classes).

Competency questions. CQ 3 (Jessica, a focus question): her request expresses three information requirements, and her preference setting yields the safety-screening HardConstraint; the capabilities that satisfy them belong to the personalized supplement recommendation, evidence verification, product selection and supplement safety screening skills, which is the answer the reasoner returns on the example data. CQ 5 (Dante, a focus question) reads the same links from the tasks of his plan: each task requires a capability, and that capability satisfies one of his information requirements or his safety-screening hard constraint.

Changes in 2.0. satisfies is split into satisfiesRequirement and satisfiesConstraint. In the ontology satisfiesConstraint now has the domain OrchestrationPlan or Capability, so a capability can satisfy a hard constraint as Version 1.1 drew it.

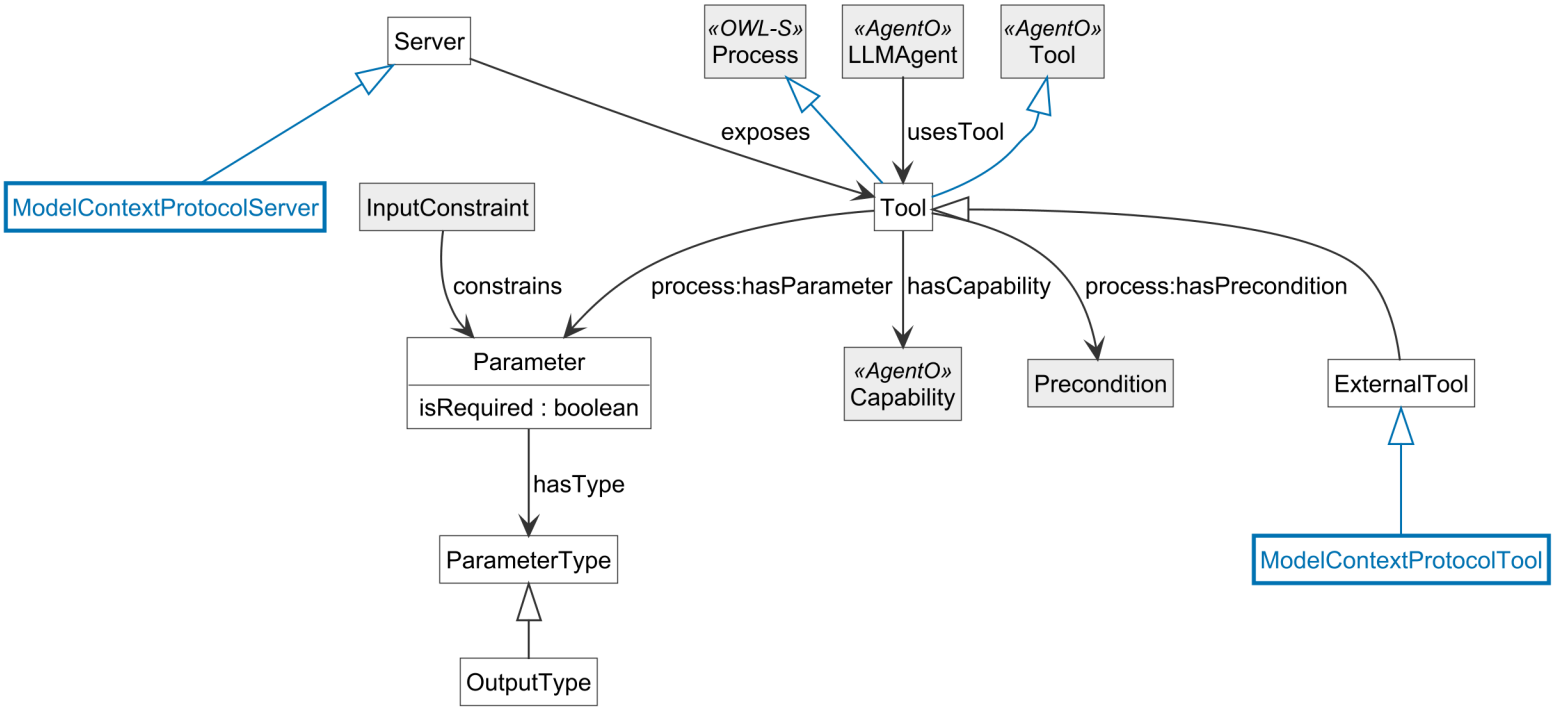


Figure 4. Tools, servers and tool interfaces

Shows. A Server exposes Tools. The PubMed, Supplement Advisor and OpenFDA servers are ModelContextProtocolServers [4] whose tools are ModelContextProtocolTools; iHerb-CLI's commands are ExternalTools reached through a command line. A Tool's interface is a set of Parameters, each with a ParameterType; OutputType types what a tool returns, isRequired marks required parameters, and an InputConstraint constrains a parameter's values. The LLMAgent uses tools.

Modeling decisions. Interface facts (parameters, types, preconditions) are kept apart from run-time facts (Figure 7a), so the harness can check a plan before running it, as the third requirement in Section II of the use case asks. Tool is a subclass of agento:Tool and of OWL-S process:Process [8], so the AgentO and OWL-S properties apply to it unchanged.

Competency questions. CQ 9 checks required parameters and preconditions before a call; alternate flow 2 lists the available tools with their capabilities.

Changes in 2.0. MCPTool and MCPServer are renamed ModelContextProtocolTool and ModelContextProtocolServer, as in the ontology. Tool's two superclasses are drawn. The figure is drawn in the shared style of the other figures.

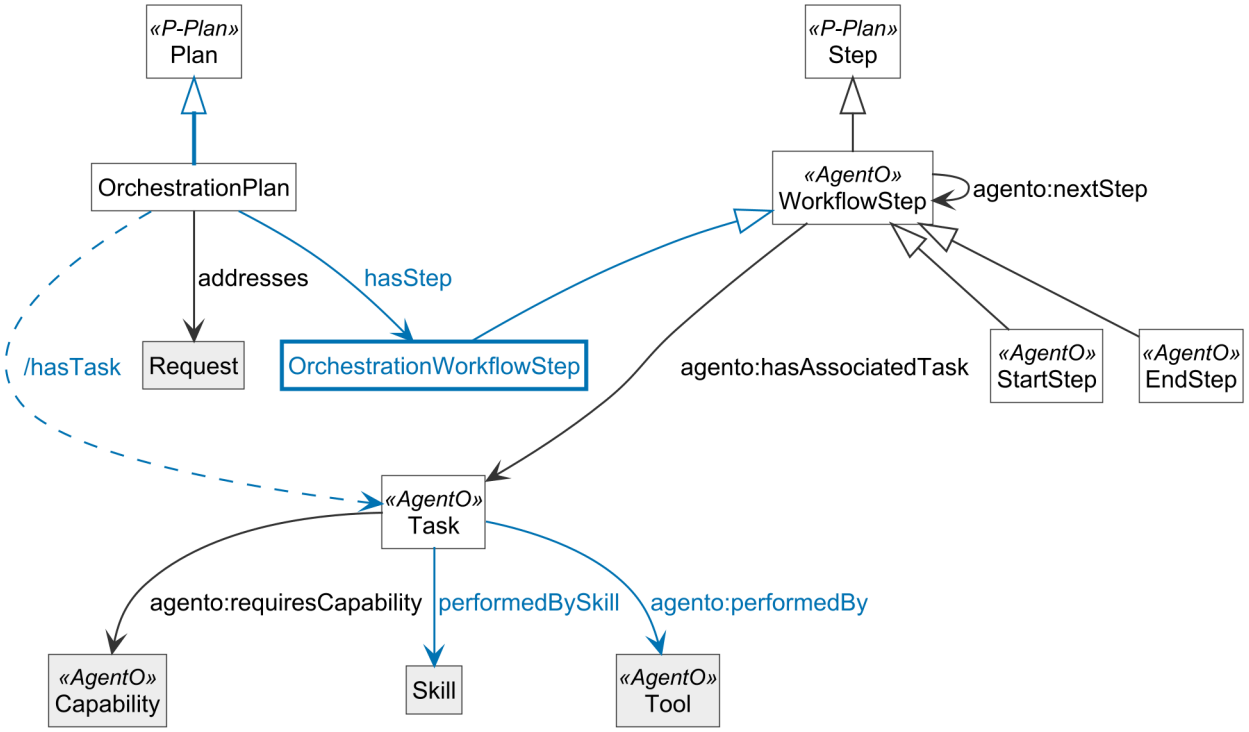


Figure 5a. Plan structure

Shows. An OrchestrationPlan is a P-Plan Plan [12]. It addresses one Request and has OrchestrationWorkflowSteps (hasStep), a kind of AgentO WorkflowStep; agento:nextStep orders the steps from a StartStep to an EndStep. Each step is associated with one Task. A Task requires one or more Capabilities and is performed either by a Skill (performedBySkill) or by a Tool (agento:performedBy). /hasTask links a plan to the tasks of its steps.

Modeling decisions. hasTask is derived: hasStep o agento:hasAssociatedTask SubPropertyOf hasTask, so the shortcut cannot disagree with the steps. A task performed by a tool reuses agento:performedBy exactly as AgentO defines it, from Task to Tool. A task performed by a skill uses the new performedBySkill, named after AgentO's performedByAgent; it is not a subproperty of agento:performedBy, because that property's range would make every Skill a Tool.

Competency questions. CQ 1 (Ken, a focus question) lists the selected plan's tasks through /hasTask, in agento:nextStep order; CQ 2 uses the same tasks in Figure 5b. CQ 5 (Dante, a focus question) follows Invocation → OrchestrationWorkflowStep → Task → agento:requiresCapability → Capability, and the performer link shows whether the capability came from a skill or directly from a tool.

Changes in 2.0. Figure 5 is split into 5a and 5b. OrchestrationPlan is stated as a subclass of pplan:Plan. The plan's steps use hasStep and OrchestrationWorkflowStep, as in the ontology. /hasTask is added as a derived shortcut. performedBy is split into agento:performedBy and performedBySkill.

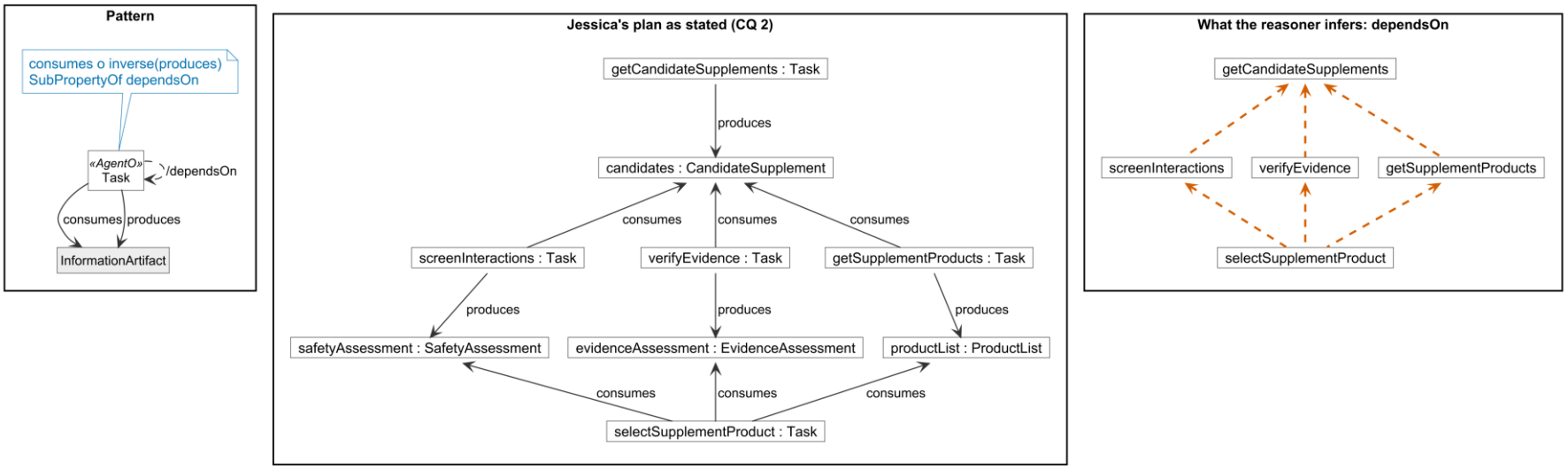


Figure 5b. Task dependencies

Shows. Left, the pattern: a Task consumes and produces InformationArtifacts, and dependsOn is derived from them. Middle, Jessica's plan as the KG stores it (CQ 2): only consumes and produces are stated. Right, the dependsOn links that the reasoner infers from the middle panel.

Modeling decisions. Dependencies are derived, not entered by hand. Task B dependsOn Task A when B consumes an artifact that A produces, written in Protégé as consumes o inverse(produces) SubPropertyOf dependsOn [14]. The KG therefore never stores a dependency that disagrees with the artifacts.

Competency questions. CQ 2 (Jessica): screenInteractions, verifyEvidence and getSupplementProducts each depend only on getCandidateSupplements, and no dependsOn links any two of them, so the three can run in parallel; selectSupplementProduct depends on all three and runs after them. With the merged ontology, Pellet infers these links from the stated ones, and they give the CQ 2 answer in the use case. CQ 4 (Dante) and CQ 8 read the order from the same inferred links.

Changes in 2.0. New figure, split from Version 1.1's Figure 5. A worked example sets the stated links beside the inferred ones, answering the presentation feedback that asked for a more explicit link between diagrams and competency questions.

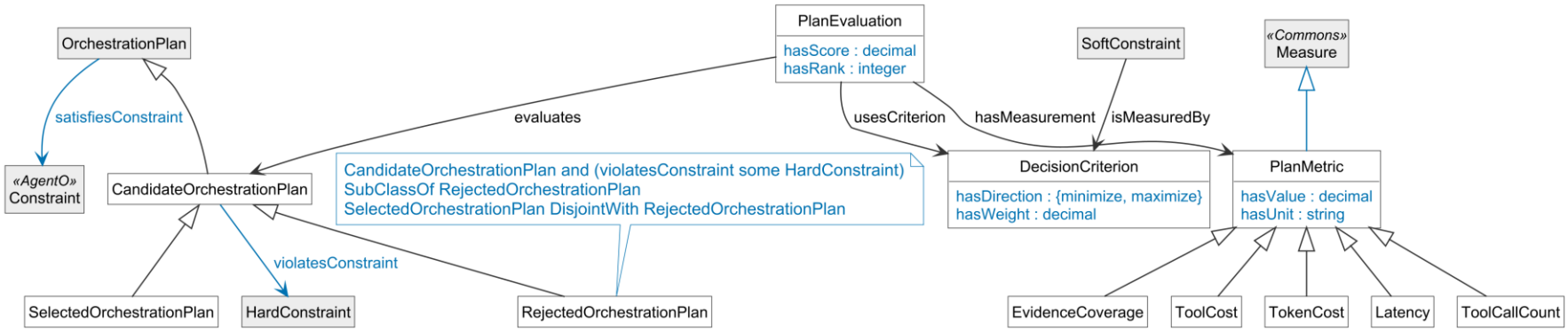


Figure 6. Plan evaluation and selection

Shows. Candidate plans for a request are compared on DecisionCriteria. A PlanEvaluation evaluates one CandidateOrchestrationPlan: it records the DecisionCriteria used, the PlanMetrics measured, and the plan's score and rank, following the n-ary relation pattern [13]. A DecisionCriterion has a direction (minimize or maximize) and a weight taken from the priority of the SoftConstraint it measures. The metrics are ToolCallCount, EvidenceCoverage, ToolCost, TokenCost and Latency, each a Measure in the sense of the OMG Commons library [18].

Modeling decisions. Selection runs in three steps, which is how conflicting criteria are handled. Hard constraints filter first: violatesConstraint links a candidate only to a HardConstraint, and the ontology classifies a candidate that violates one as a RejectedOrchestrationPlan (the general class axiom in the note). The remaining candidates are ranked by their weighted criterion scores; a soft constraint is never violated, it only lowers a plan's score and rank. Soft constraints that directly conflict (Figure 2) stop planning and go back to the user. The chosen plan is typed SelectedOrchestrationPlan and the others RejectedOrchestrationPlan; the two classes are disjoint, and the harness states both, because under the open-world assumption the KG cannot conclude that a plan was rejected just because it is not marked as selected.

Competency questions. CQ 1 (Ken, a focus question): on the example data the reasoner finds no violated hard constraint; plan 1's evaluation has score 2.00 and rank 1 (4 tool calls, $0.02 in tokens) and plan 2's has 0.00 and rank 2 (9 tool calls, $0.05), with Ken's two soft constraints at the same priority, so plan 1 is selected. CQ 6 (Olivia) compares plans A and B the same way on evidence coverage and tool calls, and CQ 7 follows Plan A's evaluation to the soft constraint it measures. CQ 4 (Dante) ranks by Latency.

Changes in 2.0. score, rank, direction, weight, value and unit are renamed hasScore, hasRank, hasDirection, hasWeight, hasValue and hasUnit. evaluates was missing from the merged ontology and is added to it. violates becomes violatesConstraint with the range HardConstraint, and the rule that a violating candidate is rejected is new. satisfiesConstraint is drawn from OrchestrationPlan, its domain. PlanMetric's superclass Measure is drawn.

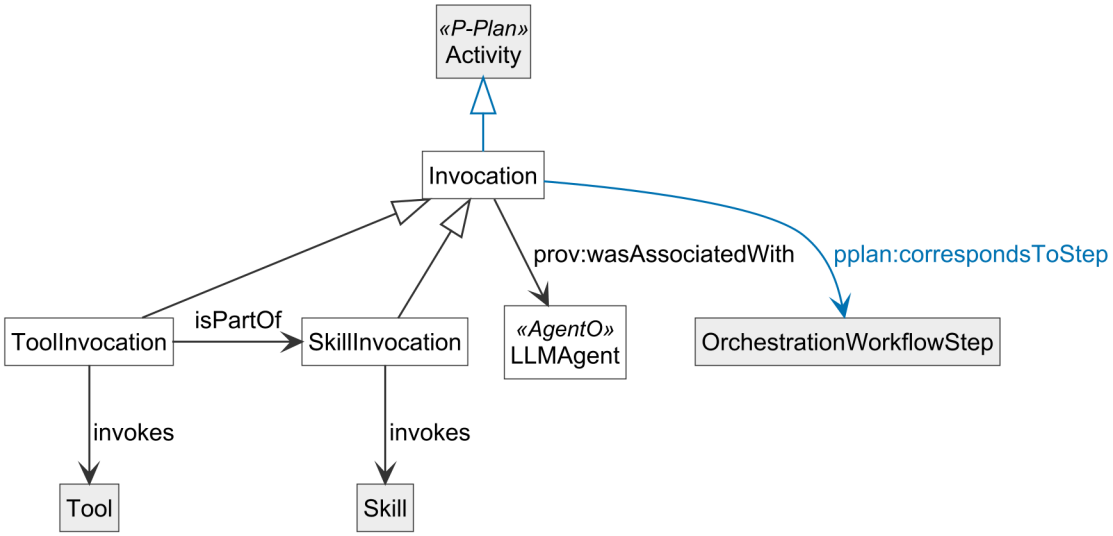


Figure 7a. Invocations

Shows. The run-time side of a plan, following P-Plan [12] and PROV-O [9]. An Invocation is a P-Plan Activity that corresponds to one OrchestrationWorkflowStep and is associated with the LLMAgent that ran it. A ToolInvocation invokes a Tool and is part of the SkillInvocation that made the call; the three OpenFDA calls in Jessica's plan, for example, are part of one Screen interactions invocation.

Modeling decisions. The plan (Figure 5a) and its execution are kept separate, so the KG can compare what was planned with what ran; reusing PROV-O and P-Plan instead of new terms keeps the record readable by existing provenance tools.

Competency questions. CQ 4 and CQ 5 (a focus question) build their explanations from these records.

Changes in 2.0. Figure 7 is split into 7a and 7b; the A5 feedback found the provenance figure crowded. Invocation's superclass pplan:Activity is drawn, and pplan:correspondsToStep now ends at OrchestrationWorkflowStep.

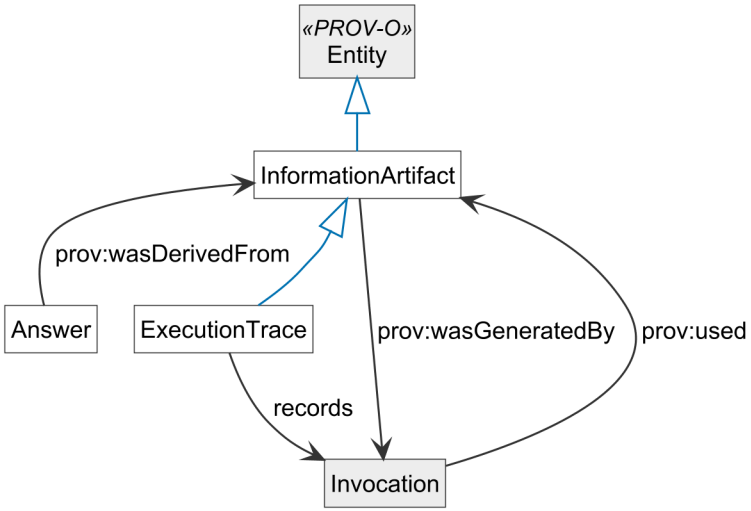


Figure 7b. Provenance of information artifacts

Shows. An InformationArtifact is a PROV-O Entity. An Invocation uses artifacts and generates new ones; the Answer is derived from the artifacts it summarizes; an ExecutionTrace is itself an information artifact and records the invocations, as basic flow step 11 requires.

Competency questions. CQ 9 checks which artifacts have been generated so far (prov:wasGeneratedBy).

Changes in 2.0. Split from Version 1.1's Figure 7. The superclasses of InformationArtifact (prov:Entity) and of ExecutionTrace (InformationArtifact) are drawn.

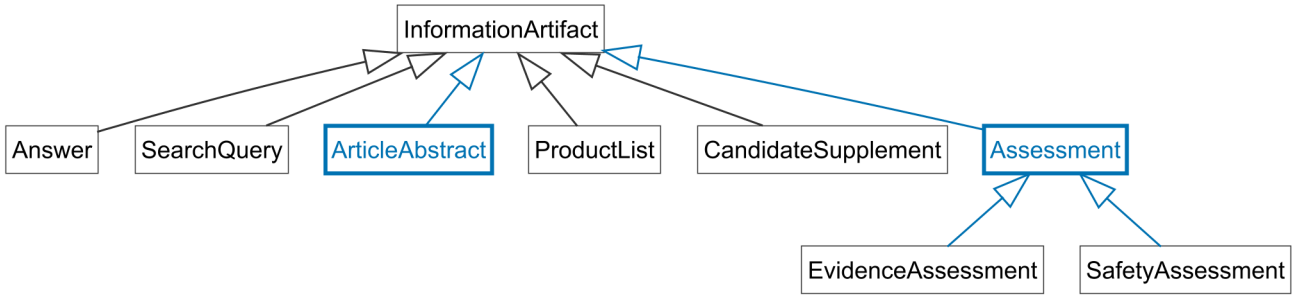


Figure 8a. Kinds of information artifacts

Shows. The kinds of artifacts that the use case flows pass between steps. A SearchQuery feeds the search tools; an ArticleAbstract is what PubMed download_articles_batch returns; a CandidateSupplement comes from the Supplement Advisor and a ProductList from iHerb-CLI; the Answer is the final response (Figure 7b). EvidenceAssessment and SafetyAssessment are Assessments, each of one CandidateSupplement (Figure 8c).

Modeling decisions. Kinds are shown here and roles in Figure 8b. Assessment groups the two artifacts that judge a candidate supplement and carries assessesSupplement for both. ArticleAbstract is added because each finding in Figure 8c needs its source.

Competency questions. CQ 3 and CQ 8 name these kinds as what skills' tasks consume and produce; CQ 2 traces them through the tasks of Figure 5b.

Changes in 2.0. Version 1.1 drew kinds and roles in one figure, with eight inheritance lines into InformationArtifact. New classes Assessment and ArticleAbstract; EvidenceAssessment and SafetyAssessment move under Assessment.

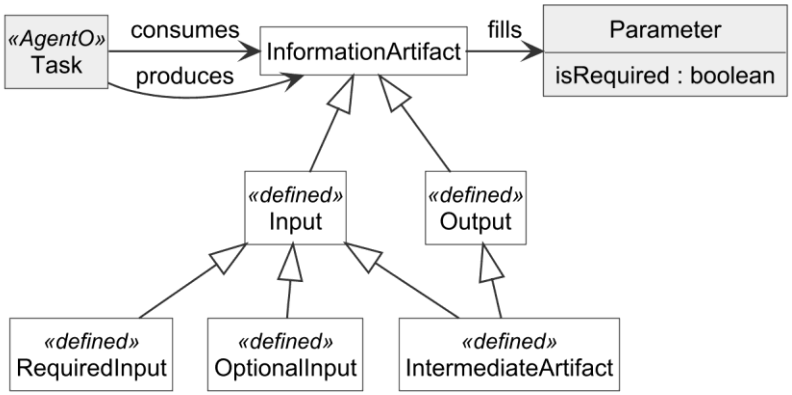


Figure 8b. Roles of information artifacts

Shows. Input, Output, RequiredInput, OptionalInput and IntermediateArtifact are roles that an artifact plays in a plan, not kinds, following the party and role pattern from class. They are «defined» classes, so the reasoner assigns them from consumes, produces and fills:

Class	Definition (Manchester syntax)
Input	`InformationArtifact and (inverse consumes some Task)`
Output	`InformationArtifact and (inverse produces some Task)`
IntermediateArtifact	`Input and Output`
RequiredInput	`Input and (fills some (Parameter and (isRequired value true)))`
OptionalInput	`Input and (fills some (Parameter and (isRequired value false)))`

Modeling decisions. Because the roles are defined, an artifact's role follows the plan instead of being entered by hand. In Jessica's plan the reasoner classifies the candidate supplements as an Output, an Input and an IntermediateArtifact.

Competency questions. CQ 9: before verifyEvidence runs, the harness checks that each RequiredInput it consumes has been generated (prov:wasGeneratedBy, Figure 7b). The system writes these records itself, so this check can use a closed-world query.

Changes in 2.0. The five role definitions are written out in Manchester syntax; fills and Parameter move here from the Version 1.1 figure.

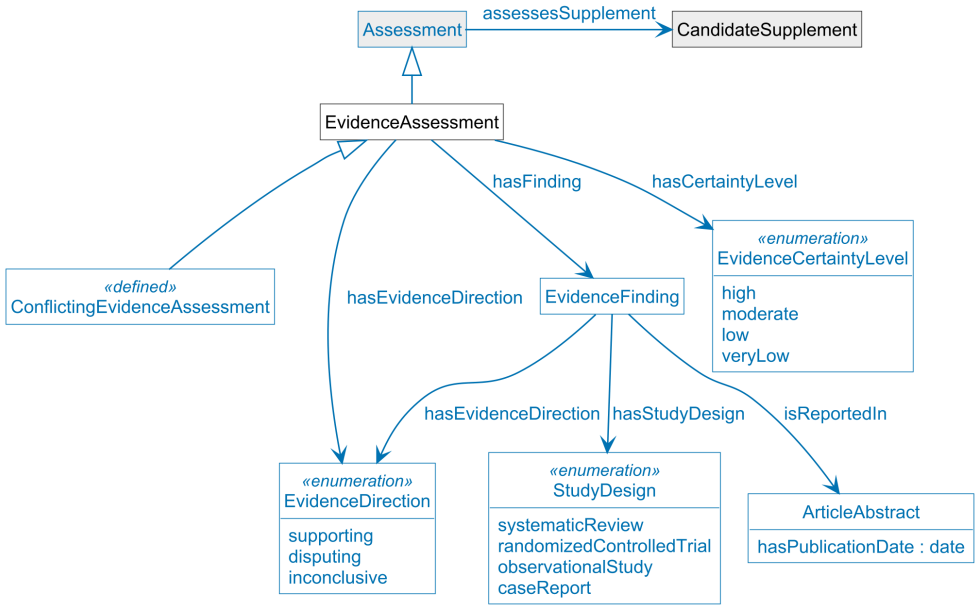


Figure 8c. Evidence assessment

Shows. How Verify evidence summarizes and weighs evidence (summarize_evidence() in the use case flows). An EvidenceAssessment assesses one CandidateSupplement and has one EvidenceFinding per PubMed article. A finding records its direction (supporting, disputing or inconclusive, from SEPIO [15]), the design of the study (MeSH publication types [16]) and the ArticleAbstract that reports it, whose publication date gives recency. The assessment states an overall direction and a certainty level (high, moderate, low or very low, from GRADE [17]).

Modeling decisions. A finding is an n-ary relation [13]: direction, design and source belong to one article's result, not to the supplement. The value sets are classes whose members are the listed individuals, so queries can compare them. Conflicting evidence is explicit: ConflictingEvidenceAssessment is defined as an EvidenceAssessment with at least one supporting and at least one disputing finding, and the reasoner classifies it. summarize_evidence() sets the certainty level the way GRADE starts it [17]: from the best design among the supporting findings (systematic review or randomized controlled trial high, observational study low, case report very low), one level lower if the assessment is conflicting, and one level lower if the newest supporting article is more than ten years old.

Competency questions. Jessica asks for high-quality scientific evidence (usage scenario 1); her recommendation keeps only supplements whose assessment has high or moderate certainty. EvidenceCoverage in Figure 6 (CQ 6, Olivia) is the share of candidate supplements whose assessment has a supporting overall direction; Plan A's second PubMed pass raises it from 0.70 to 1.00. For Dante's request to summarize the evidence for each supplement (CQ 4), the answer gives each assessment's direction and certainty, with its findings as the explanation.

Changes in 2.0. New figure. It answers the presentation question on how evidence is summarized and weighed, including conflicting evidence, recency and quality.

How the model answers the competency questions

Bold rows are the three focus questions that scope the Assignment 6 ontology; their answers were checked with the Pellet reasoner on example individuals that follow the usage scenarios (see the use case, Section VII).

CQ	Scenario	Figures	Path through the model
1. What orchestration should the agent follow? (focus)	**Ken**	**2, 3b, 5a, 6**	**PreferenceSetting ← prov:wasDerivedFrom ← SoftConstraints (few tool calls, low token use) → isMeasuredBy → DecisionCriteria; no HardConstraint violated → PlanEvaluations evaluate the candidates on ToolCallCount and TokenCost → hasScore 2.00 vs 0.00, hasRank 1 vs 2 → SelectedOrchestrationPlan → derived /hasTask lists its tasks in nextStep order**
2. Which steps can run in parallel?	Jessica	5a, 5b, 8b	Plan → hasStep → OrchestrationWorkflowStep → agento:hasAssociatedTask → Task (derived /hasTask); tasks consume and produce artifacts → derived dependsOn → screenInteractions, verifyEvidence and getSupplementProducts depend only on getCandidateSupplements, so no dependsOn links them
3. What skills are required? (focus)	**Jessica**	**2, 3b**	**Request → expresses → InformationRequirement ← satisfiesRequirement ← Capability; PreferenceSetting ← prov:wasDerivedFrom ← HardConstraint ← satisfiesConstraint ← Capability; Skill → hasCapability → those capabilities**
4. Why were capabilities invoked in this order?	Dante	2, 5b, 6, 7a, 8c	dependsOn gives the order; the safety-screening HardConstraint is satisfied by the screening capability; the SoftConstraint “minimize execution time” is measured by a Latency criterion that ranks the parallel plan first; invocation records and the two EvidenceAssessments supply the explanation
5. Why was each capability chosen? (focus)	**Dante**	**3b, 5a, 7a**	**Selected plan → derived /hasTask → Task → agento:requiresCapability → Capability → satisfiesRequirement (a requirement the request expresses) or satisfiesConstraint (the safety-screening HardConstraint); performedBySkill names the skill; Invocation → pplan:correspondsToStep gives the run-time record**
6. Why was plan B selected over plan A?	Olivia	2, 6, 8c	No HardConstraint violated (violatesConstraint) → PlanEvaluations evaluate A and B: EvidenceCoverage 1.00 vs 0.70, ToolCallCount 8 vs 3; equal hasPriority → equal hasWeight → hasScore 1.00 vs 1.70, hasRank 2 vs 1 → B is SelectedOrchestrationPlan, A is RejectedOrchestrationPlan
7. Which constraints were violated, and which soft constraints lowered a plan’s rank?	Olivia, Jessica	2, 6	Plan A violates no HardConstraint; its PlanEvaluation → usesCriterion → tool-usage criterion ← isMeasuredBy ← SoftConstraint “minimize external tool calls”. Jessica’s plan without screening → violatesConstraint → HardConstraint → classified RejectedOrchestrationPlan
8. What skills depend on other skills?	—	5a, 5b	Skill ← performedBySkill ← Task; tasks’ consumes and produces → derived dependsOn → the evidence and screening skills depend on the recommendation skill
9. Are the prerequisites satisfied before a skill runs?	—	3a, 4, 7b, 8b	RequiredInputs the skill’s task consumes and the skill’s Preconditions vs. artifacts generated so far (prov:wasGeneratedBy) → executable or blocked, plus the task that produces what is missing

References

Numbers [1]–[14] follow Section IX of the use case; [15]–[18] are new in Version 2.0.

[1] I. O. Mulang', J. Thaller, T. Trivedi, L. Heling, and F. Sasaki, "Representing agentic tools in knowledge graphs for structure-aware tool discovery under tool overload," in Proc. 2nd Workshop on Knowledge Graphs and Agentic Systems Interplay (NORA 2026), co-located with IJCAI-ECAI 2026, Bremen, Germany, 2026. [Online]. Available: https://openreview.net/forum?id=7MVoH9Y3mi

[2] A. Ekelhart, K. Kurniawan, F. J. Ekaputra, and E. Kiesling, "AgentO: An ontology for modeling agentic AI systems," in The Semantic Web: 23rd European Semantic Web Conference, ESWC 2026, Proceedings, Part II, Lecture Notes in Computer Science, vol. 16550, M. Acosta et al., Eds. Cham, Switzerland: Springer, 2026, pp. 298–320, doi: 10.1007/978-3-032-25159-6_16.

[3] K. Kurniawan and F. J. Ekaputra, "AgentO: An ontology for modeling agentic AI systems," ontology release v0.2, Zenodo, Jan. 22, 2026, doi: 10.5281/zenodo.18342625. [Online]. Available: https://w3id.org/agentic-ai/onto

[4] Model Context Protocol Project, "Model Context Protocol specification," revision 2026-07-28, LF Projects, LLC, Jul. 28, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28

[5] X. Li, Y. Liu, W. Chen, B. You, Z. Di, Y. He, et al., "SkillsBench: Benchmarking how well agent skills work across diverse tasks," arXiv:2602.12670, Feb. 2026. [Online]. Available: https://arxiv.org/abs/2602.12670

[8] D. Martin et al., "OWL-S: Semantic markup for web services," W3C Member Submission, Nov. 22, 2004. [Online]. Available: https://www.w3.org/submissions/OWL-S/

[9] T. Lebo, S. Sahoo, and D. McGuinness, Eds., "PROV-O: The PROV ontology," W3C Recommendation, Apr. 30, 2013. [Online]. Available: https://www.w3.org/TR/prov-o/

[12] D. Garijo and Y. Gil, "The P-Plan ontology," version 1.3, Mar. 12, 2014. [Online]. Available: http://purl.org/net/p-plan

[13] N. Noy and A. Rector, Eds., "Defining N-ary relations on the Semantic Web," W3C Working Group Note, Apr. 12, 2006. [Online]. Available: https://www.w3.org/TR/swbp-n-aryRelations/

[14] J. Bao, E. F. Kendall, D. L. McGuinness, and P. F. Patel-Schneider, Eds., "OWL 2 Web Ontology Language quick reference guide (second edition)," W3C Recommendation, Dec. 11, 2012. [Online]. Available: https://www.w3.org/TR/owl2-quick-reference/

[15] Monarch Initiative, "Scientific Evidence and Provenance Information Ontology (SEPIO)," release 2023-06-13, OBO Foundry. [Online]. Available: http://purl.obolibrary.org/obo/sepio.owl

[16] U.S. National Library of Medicine, "Medical Subject Headings (MeSH): publication types Systematic Review (D000078182), Randomized Controlled Trial (D016449), Observational Study (D064888), and Case Reports (D002363)." [Online]. Available: https://meshb.nlm.nih.gov/ (accessed Oct. 7, 2026).

[17] H. Balshem et al., "GRADE guidelines: 3. Rating the quality of evidence," J. Clin. Epidemiol., vol. 64, no. 4, pp. 401–406, Apr. 2011, doi: 10.1016/j.jclinepi.2010.07.015.

[18] Object Management Group, "Commons Ontology Library (Commons), Version 1.2," QuantitiesAndUnits module, version 20240801. [Online]. Available: https://www.omg.org/spec/Commons/